Real-time Collaborative Code Editor — Low-Level Architecture (v1)

> Goal: Ship an MVP that supports multi-user, low-latency collaborative editing with presence, selections, and basic auth. Optimize for correctness, observability, and incremental scalability.

---

0) Tech Choices (Opinionated)

- **Editor**: Monaco
- **Collab Engine**: **Yjs (CRDT)** with awareness protocol (presence)
- **Transport**: WebSocket (wss) using a custom provider (y-websocket) with auth
- **Backend**: Node.js (Fastify) + ws
- **State Fanout**: Redis Pub/Sub (for horizontal scaling of socket nodes)
- **Persistence**:
- Document state: y-leveldb (single node) → **y-redis** (ephemeral) → **Yjs persistence adapter w/ PostgreSQL** for durability (via y-indexeddb on client for offline)
- Metadata: PostgreSQL (users, documents, permissions, audit)
- Assets (images, file attachments): S3-compatible object store (e.g., MinIO)
- **Auth**: OAuth (GitHub/Google) → JWT (short-lived) + Refresh tokens (httpOnly cookie)
- **Gateway**: Nginx/Cloudflare (TLS termination, rate-limit)
- **Observability**: OpenTelemetry (traces/metrics/logs) + Prometheus + Grafana

> Why CRDT/Yjs? Offline/edit-anywhere UX, simpler client conflict handling, strong community, great Monaco bindings. OT (e.g., ShareDB) is also viable; see §11 for alternatives.

---

1) High-Level Component Diagram (Textual)

Client (Web App) - Monaco Editor + Yjs binding (y-monaco) - Presence UI (avatars, cursors) via Yjs Awareness - Auth module (OAuth → token storage) - Document router (join/leave rooms)

API Gateway - TLS termination, CORS, websocket upgrade proxy, rate limiting

Collab Service (WS Nodes, stateless) - AuthZ on connect - Hosts Yjs docs in-memory (doc maps) per room - Broadcast deltas/awareness updates - Publishes/consumes Redis channels for cross-node sync

Persistence Service - Periodic Yjs snapshots + updates → Postgres (via persistence adapter) - Metadata CRUD REST (documents, membership, shares) - Object storage pre-signed URL service

Infra - Redis cluster (Pub/Sub) - Postgres (HA later) - Object Store (S3/MinIO) - Telemetry stack (OTel collector, Prometheus, Grafana, Loki)

---

2) Data Model (PostgreSQL)

```sql
-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  display_name TEXT,
  avatar_url TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Documents (logical metadata)
CREATE TABLE documents (
  id UUID PRIMARY KEY,
  owner_id UUID REFERENCES users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  language TEXT NOT NULL DEFAULT 'typescript',
  visibility TEXT NOT NULL DEFAULT 'private', -- private|link|org
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Permissions (RBAC)
CREATE TYPE doc_role AS ENUM ('owner','editor','viewer');
CREATE TABLE document_members (
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  document_id UUID REFERENCES documents(id) ON DELETE CASCADE,
  role doc_role NOT NULL,
  added_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, document_id)
);

-- Shares (link-based access)
CREATE TABLE document_links (
  id UUID PRIMARY KEY,
  document_id UUID REFERENCES documents(id) ON DELETE CASCADE,
  role doc_role NOT NULL,
  token TEXT UNIQUE NOT NULL, -- random slug
  expires_at TIMESTAMPTZ
);

-- Audit (minimal)
CREATE TABLE access_audit (
  id BIGSERIAL PRIMARY KEY,
  document_id UUID,
  user_id UUID,
  action TEXT,
  ip INET,
```

```
  ts TIMESTAMPTZ DEFAULT now()
);
```

Yjs Persistence (conceptual): - `yjs_docs(document_id UUID, snapshot BYTEA, clock BIGINT, updated_at TIMESTAMPTZ)` - `yjs_updates(document_id UUID, update BYTEA, seq BIGINT, ts TIMESTAMPTZ)`

---

3) Protocols & Message Schemas

3.1 WebSocket URL

```
wss://api.example.com/collab?docId=<uuid>&token=<jwt>
```

- **Auth**: JWT must include `sub` (user id), `scopes`, optional `link_token` claim for link-based access.

3.2 On-Connect Flow

1. Gateway upgrades to WS; forwards to a WS node (sticky by `docId` hash or cookie).
2. WS node verifies JWT (public JWK) and authorizes role for `docId` via Postgres cache.
3. Node loads Yjs doc from memory or reconstructs from last snapshot + incremental updates from Postgres.
4. Node sends `INIT_ACK` with current awareness state & version clock (optional envelope around y-protocol messages).

3.3 Awareness / Presence (Yjs Awareness)

- Encodes user status (cursor pos, selection, name, color, idle/activity). Broadcast via WS; throttled to ~10–20 Hz.

3.4 Application Envelopes (optional)

```
// Client → Server
{
  "t": "PING" | "AWARENESS" | "CUSTOM_EVENT",
  "ts": 1730000000,
  "payload": { /* opaque */ }
}

// Server → Client
{
  "t": "PONG" | "ACK" | "ERROR",
  "ts": 1730000001,
  "payload": { "code": "UNAUTHORIZED" }
}
```

Note: Yjs update frames are binary; we multiplex them on the same socket (no need for custom JSON envelope unless adding non-Yjs events).

4) REST API (Metadata, Auth, Storage)

```
POST   /v1/auth/oauth/callback                -> sets httpOnly refresh
cookie, returns short-lived JWT
GET    /v1/users/me                           -> profile
POST   /v1/documents                          -> create {title, language}
GET    /v1/documents/:id                      -> metadata
PATCH  /v1/documents/:id                      -> title, visibility,
language
DELETE /v1/documents/:id                      -> delete (soft-delete
optional)
POST   /v1/documents/:id/members              -> add collaborator {userId,
role}
DELETE /v1/documents/:id/members/:userId      -> remove collaborator
POST   /v1/documents/:id/links                -> create link {role,
expiresAt}
GET    /v1/storage/upload-url?name=&type=     -> pre-signed URL (S3) for
assets
```

Auth: Bearer JWT for API; WS uses query param or `Sec-WebSocket-Protocol: bearer,<token>` header.

---

5) Scaling Topology

- **WS Shards**: N replicas behind gateway; each holds many docs. Rebalance by consistent hashing of `docId` → shard.
- **Cross-Node Sync**: Redis Pub/Sub per `docId` channel to relay Yjs updates when same document is open on multiple shards.
- **Persistence**:
- Snapshot every X updates or Y seconds; truncate old updates after snapshot.
- Background compactor job.
- **Hot-doc Eviction**: LRU cache; write last snapshot to Postgres; evict doc from memory when idle.

---

6) Failure Modes & Recovery

- **Node Crash**: Clients auto-reconnect via gateway to another shard. New shard reconstructs doc state from latest snapshot + updates.
- **Network Partition**: CRDT ensures eventual convergence. On reconnect, client syncs missing updates; server persists.
- **Redis Outage**: Single-doc edits on one shard continue; multi-shard fanout degrades (warn users). Queue updates locally and replay on recovery.
- **Postgres Lag/Outage**: Continue realtime (in-memory) for active docs; back-pressure persistence, alert SRE; reject new docs if RPO/RTO at risk.

---

7) Security Model

- **Transport**: TLS 1.2+, secure cookies, HSTS
- **AuthZ**: Role checks on every WS join; server enforces read-only for viewers (ignore mutating updates)
- **Isolation**: Per-doc rate limits; per-IP connection limits; message size caps; schema validation for app envelopes
- **Sandbox**: Any code execution features must run in isolated containers/VMs; no eval in browser
- **Secrets**: JWT signing keys in KMS; rotate regularly
- **Audit**: Log joins/leaves, role changes, link access

---

8) Performance Targets

- **Tail Latency (P95)**: < 120 ms for remote update visible on peer
- **Awareness Broadcast**: ≤ 20 Hz per user, coalesced
- **Doc Size**: Smooth editing up to 5–10 MB text; snapshot compaction > 1 MB
- **Fanout**: Handle 50 concurrent editors per doc, 1k viewers across shards

---

9) Observability

- **Metrics**: active_connections, updates_in/out, p95_apply_latency, snapshot_duration, redis_backlog, ws_reconnects
- **Traces**: connect → authorize → load snapshot → join room; update ingest → persist → pubsub fanout
- **Logs**: structured JSON; privacy-safe (no document content)
- **Alerts**: High reconnect rates, snapshot failures, Redis lag, Postgres write errors

---

10) Client Architecture (React + Monaco)

```
<App>
  <AuthProvider>
    <Router>
      /doc/:id → <EditorPage>
         ├─ useDocConnection(docId, token)  // connects WS, returns ydoc,
awareness
         ├─ <MonacoEditor bind={ydoc} />    // y-monaco
         ├─ <PresencePanel awareness />
         ├─ <Outline/Tree optional />
         └─ <Chat optional />
    </Router>
  </AuthProvider>
</App>
```

Key hooks: - `useDocConnection`: manages connect/reconnect, backoff, auth renewal, visibility-change pauses - `useLatencyMeter`: display round-trip time; feed metrics

11) OT Alternative (ShareDB) — Brief

- **Pros**: Mature OT, central transform logic (server-authoritative), smaller updates for some workloads
- **Cons**: Harder offline support; server CPU for transforms; more custom logic
- **If chosen**: Add `ops` collection in Mongo/Postgres; implement transform/compose; persistent oplog; snapshots + compaction; optimistic UI with server ack

---

12) Pseudocode (Server Skeleton)

```
// Fastify + ws (simplified)
const rooms = new Map<string, Room>();

function getRoom(docId) {
  let r = rooms.get(docId);
  if (!r) {
    r = new Room(docId, redis, postgres, persistence);
    rooms.set(docId, r);
  }
  return r;
}

fastify.get('/collab', { websocket: true }, async (conn, req) => {
  const { docId } = req.query;
  const user = await verifyJWT(req.query.token);
  authorize(user, docId);
  const room = getRoom(docId);
  room.join(conn.socket, user);
});

class Room {
  constructor(id, redis, pg, persistence) {
    this.id = id;
    this.ydoc = new Y.Doc();
    this.awareness = new awarenessProtocol.Awareness(this.ydoc);
    // load snapshot + updates → apply
    persistence.loadInto(this.ydoc, id);
    // subscribe redis channel for remote updates
    redis.subscribe(`doc:${id}`, (binaryUpdate) => Y.applyUpdate(this.ydoc,
binaryUpdate));
    // on local update → persist + publish
    this.ydoc.on('update', (u) => {
      persistence.append(id, u);
      redis.publish(`doc:${id}`, u);
    });
  }
  join(ws, user) {
```

```
    // hook y-websocket provider here (bind Yjs to ws)
    setupYWebSocket(ws, this.ydoc, this.awareness, user);
  }
}
```

---

13) Testing Strategy

- **Unit**: CRDT update apply; permission checks; JWT verification
- **Property-based**: Random edit sequences converge across N clients
- **Soak tests**: 24h editing with bots; measure memory growth; compaction efficacy
- **Chaos**: Kill WS node; network packet loss; Redis restart; Postgres failover

---

14) Migration & Backward Compatibility

- Version `doc.protocolVersion` in persistence
- Feature flags for new editor features; client capability negotiation on connect

---

15) Backlog (MVP → v1)

1. MVP sockets + Yjs + Monaco binding, single node, in-memory only
2. Add Postgres persistence (snapshots + updates)
3. Add Redis pub/sub for multi-node
4. OAuth login + RBAC + share links
5. Presence UI + selection colors
6. Observability baseline (metrics + dashboards)
7. Rate-limits + security hardening
8. CDN & caching for static assets
9. Optional: inline AI assistant (refactor hints)

---

16) Non-Functional Requirements (Initial)

- **Availability**: 99.5% (MVP), path to 99.9%
- **RPO**: ≤ 5s (loss of last few updates acceptable); **RTO**: ≤ 1 min
- **Privacy**: No content in logs/metrics; data encrypted at rest

---

17) Cost Notes (MVP)

- 1× small VM for WS/API, 1× small VM for Postgres, managed Redis (or single node) → <$60/mo on budget cloud; add CDN later.

This v1 is intentionally pragmatic: start simple (single node + snapshots), then layer in Redis sharding, observability, and security as you approach multi-doc, multi-team scale.